REFACTORY ACADEMY · APPLICATION SECURITY MODULE · DAY 2 OF 5

SQL, SQL Injection & Input Validation

From what a database is, to breaking a login form, to defending it properly

JAVASCRIPT / NODE + PYTHON / DJANGO TRACKS

Learning Objectives

By the end of today's class, you will be able to:

- Explain what a database, a table, a row and a column are, and read a simple SQL query
- Explain, in your own words, how SQL Injection works and why string-built queries are dangerous
- Demonstrate a real SQL Injection attack on a deliberately vulnerable login form
- Fix that same login form using parameterised queries or an ORM

- Add input validation and sanitisation on both the UI and the backend of a form

1. What Is a Database, and What Is SQL?

What Is a Database?

> **Definition:** A database stores data in tables, the same way a spreadsheet does. Each table has columns (the fields, e.g. username) and rows (the records, e.g. one technician). OAS Bay would keep separate tables for technicians, parts, customers and services, all inside one database.

What Is SQL?

> **Definition:** SQL stands for Structured Query Language. It's the language you use to talk to a database: to ask it questions ("give me every technician named kato") and to give it instructions ("add a new technician", "delete this row"). You don't touch the table directly, you write a SQL statement, and the database does the work.

Think of it like ordering at a counter instead of walking into the kitchen yourself. You don't go pull the file out of the cabinet; you ask for it in a specific way, and the database hands it back. SQL is that specific way of asking.

An OAS Bay Table: technicians

id	username	password	role
1	kato	kato2024	senior_technician
2	amina	amina2024	technician

3	admin	admin123	admin

Each row is one technician. Each column is one fact about them. We will use this exact table for today's demo, and store passwords in plain text on purpose, for now, so you can see why that's a problem too.

The 4 Commands You Need Today

Command	What It Does	Example
SELECT	Read/find rows	SELECT * FROM technicians WHERE username = 'kato';
INSERT	Add a new row	INSERT INTO technicians (username, password) VALUES ('juma', 'pass123');
UPDATE	Change existing rows	UPDATE technicians SET password = 'newpass' WHERE id = 1;
DELETE	Remove rows	DELETE FROM technicians WHERE id = 3;

Text vs Numbers: Why Those Quote Marks Matter

Look closely at the examples above: 'kato' has quote marks around it, but id = 1 does not. That's not random, it's a rule.

- Text (called a string) always goes inside single quotes: 'kato', 'kato2024', 'senior_technician'
- Numbers never need quotes: id = 1, price = 79000
- The database uses that opening and closing quote to know exactly where a piece of text starts and ends

> **Keep this in mind:** that single quote character is the exact thing an attacker will target in Section 2. If they can sneak their own quote mark into your query, they can end your text early and start writing their own SQL. This is the single most important idea in today's class.

Comments in SQL

A comment is text the database ignores completely, it's there for humans to read, not for the database to run. SQL has two comment styles:

```
-- Two dashes comment out everything to the end of the line
SELECT * FROM technicians; -- this note is ignored by the database

/* A slash-star comment
   can span multiple lines,
   and ends with a star-slash */
SELECT * FROM technicians;
```

> **Why this matters today:** -- is exactly what an attacker uses to silently delete the rest of a real query, so it never runs. You'll see ' OR '1'='1' -- again in Section 2, and '; DROP TABLE users; -- in your Day 2 assignment. Once you know -- means "ignore everything after this", both attacks stop looking like magic.

> **Read it out loud:** "SELECT star FROM technicians WHERE username equals kato" means "give me every column, from the technicians table, but only the row where username is kato." That WHERE clause is the part we're going to attack today.

2-Minute Class Exercise

In the Zoom chat: write the SQL query that finds the technician with username 'amina'. Then write the query that would find every technician whose role is 'admin'. Post both when you're done, we'll check answers together before moving on.

2. What Is SQL Injection?

> **Definition:** SQL Injection happens when user input is pasted directly into a SQL query string, instead of being treated as pure data. If an attacker's input contains SQL syntax, the database will run it, not just search for it.

How a Normal Login Query Is Built

```
-- The app builds this string using the username and password the
user typed:
SELECT * FROM technicians
WHERE username = 'kato' AND password = 'kato2024';

-- If a row comes back, the login succeeds.
```

What Happens If the Attacker Types SQL Instead of a Password?

Suppose the attacker types this into the username field, and leaves the password field empty:

```
' OR '1'='1
```

If the app just glues strings together, the query the database actually receives becomes:

```
SELECT * FROM technicians
WHERE username = '' OR '1'='1' AND password = '';

-- '1'='1' is ALWAYS true, for every single row.
-- The database returns every technician, including admin, with no
password needed.
```

VULNERABLE The database can no longer tell the difference between "data the user typed" and "instructions to run", because both were mixed into one plain string.

OWASP: *A03, Injection (SQL Injection)*

3. Practical: Break, Then Fix, a Login Query

We'll run this for real, on your own machine, not just read it. Node.js gets a plain SQLite file via the sqlite3 package. Django already ships with SQLite as its default database, so there's nothing extra to install for the database itself. Same technicians table, same attack, same fix, two languages.

> **Recommended tool:** install DB Browser for SQLite before class, from https://sqlitebrowser.org/dl/ (free, Windows/Mac/Linux). Open oasbay.sqlite (Node.js) or db.sqlite3 (Django) in it during the demo, click the Browse Data tab, and keep it visible next to the browser, so you can literally watch rows appear when the attack succeeds, not just read a response on the page. It won't auto-refresh: click File > Revert Changes (or reopen the file) each time you want it to show the latest data.

Step 0 - Local Setup: JavaScript / Node.js Track

In a terminal:

```
mkdir oasbay-day2 && cd oasbay-day2
npm init -y
npm install express sqlite3
```

Create one file, server.js, with the full app: the database, a login form, and a spot for the /login route we'll edit through the rest of this section.

JAVASCRIPT / NODE.JS · server.js

```
// server.js
const express = require('express');
const sqlite3 = require('sqlite3').verbose();
const fs = require('fs');
```

```
const app = express();
app.use(express.urlencoded({ extended: true }));

// Start from a clean file every time you restart, so the demo is
repeatable
if (fs.existsSync('./oasbay.sqlite'))
fs.unlinkSync('./oasbay.sqlite');

// Create and seed a file-based SQLite database (so DB Browser for
SQLite can open it too)
const db = new sqlite3.Database('./oasbay.sqlite');
db.serialize(() => {
  db.run(`CREATE TABLE technicians (
    id INTEGER PRIMARY KEY,
    username TEXT,
    password TEXT,
    role TEXT
  )`);
  db.run(`INSERT INTO technicians (username, password, role) VALUES
    ('kato', 'kato2024', 'senior_technician'),
    ('amina', 'amina2024', 'technician'),
    ('admin', 'admin123', 'admin')`);
});

// A simple login form to test with
app.get('/', (req, res) => {
  res.send(`
    <form method="POST" action="/login">
      <input name="username" placeholder="username"><br>
      <input name="password" type="password"
placeholder="password"><br>
      <button type="submit">Log In</button>
```

```
    </form>
  `);
});

// >>> the /login route from Step 1 goes here <<<

app.listen(3000, () => console.log('Running at
http://localhost:3000'));
```

Run it with node server.js, then open http://localhost:3000 in the browser. Leave it running, we'll edit the /login route in place for the rest of this section.

Step 0 - Local Setup: Python / Django Track

In a terminal:

```
python -m venv venv
source venv/bin/activate        # Windows: venv\Scripts\activate
pip install django
django-admin startproject oasbay .
python manage.py startapp accounts
```

> **What just happened?** startproject created your overall Django project, called oasbay. startapp created one app inside it, called accounts. A Django project is the whole site; an app is one self-contained feature inside it, e.g. accounts (logins), parts (inventory), bookings (services). We're calling this one accounts because it will hold our technician login and authentication logic. You'll see a new accounts/ folder appear, with models.py, views.py and other files already inside it.

Creating an app doesn't turn it on by itself, Django needs to be told it exists. Open oasbay/settings.py (Django already created this file for you) and find the INSTALLED_APPS list near the top. Add 'accounts' as the last line inside it:

PYTHON / DJANGO · oasbay/settings.py

```
# oasbay/settings.py  (edit the file Django already created, don't
create a new one)
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
```

```
    'django.contrib.staticfiles',
    'accounts',              # <-- add this line, this turns your
app on
]
```

Django already ships with SQLite as its default database, so no further configuration is needed there.

PYTHON / DJANGO · accounts/models.py

```
# accounts/models.py
from django.db import models

class Technician(models.Model):
    username = models.CharField(max_length=50)
    password = models.CharField(max_length=50)
    role = models.CharField(max_length=50)
```

```
python manage.py makemigrations accounts
python manage.py migrate
```

Seed the same 3 technicians using the Django shell:

```
python manage.py shell
>>> from accounts.models import Technician
>>> Technician.objects.create(username='kato', password='kato2024',
role='senior_technician')
>>> Technician.objects.create(username='amina',
password='amina2024', role='technician')
>>> Technician.objects.create(username='admin',
password='admin123', role='admin')
>>> exit()
```

PYTHON / DJANGO · urls.py

```
# accounts/urls.py  (new file)
```

```
from django.urls import path
from . import views

urlpatterns = [
    path('', views.login_page),
    path('login/', views.login_view),
]

# oasbay/urls.py  (edit the file Django already created)
from django.urls import path, include

urlpatterns = [
    path('', include('accounts.urls')),
]
```

PYTHON / DJANGO · accounts/views.py

```
# accounts/views.py
from django.http import HttpResponse
from django.db import connection
from django.views.decorators.csrf import csrf_exempt
from .models import Technician

def login_page(request):
    return HttpResponse('''
        <form method="POST" action="/login/">
            <input name="username" placeholder="username"><br>
            <input name="password" type="password"
placeholder="password"><br>
            <button type="submit">Log In</button>
        </form>
    ''')

# >>> the login_view route from Step 1 goes here <<<
```

Run it with python manage.py runserver, then open http://localhost:8000 in the browser. We'll fill in login_view for the rest of this section.

> **Note:** login_view is wrapped with @csrf_exempt purely to keep this classroom demo simple, a plain HTML form with no CSRF token would otherwise be blocked. Django's CSRF protection is a good thing, never disable it like this in a real project.

Step 1 - The Vulnerable Login (String Concatenation)

Paste this into the spot marked above in each file.

JAVASCRIPT / NODE.JS · server.js

```
// goes inside server.js, above app.listen()
app.post('/login', (req, res) => {
  const { username, password } = req.body;
  const query = "SELECT * FROM technicians WHERE username='" +
username +
                "' AND password='" + password + "'";
  console.log('Running query:', query); // watch this in the
terminal during the demo
  db.get(query, (err, row) => {
    if (row) return res.send('Welcome ' + row.username);
    return res.send('Invalid login');
  });
});
```

PYTHON / DJANGO · accounts/views.py

```
# goes inside accounts/views.py, replacing the marker comment
@csrf_exempt
def login_view(request):
    username = request.POST.get('username')
    password = request.POST.get('password')
    query = "SELECT * FROM technicians WHERE username='" + username
+ \
            "' AND password='" + password + "'"
    print('Running query:', query)  # watch this in the terminal
during the demo
    with connection.cursor() as cursor:
        cursor.execute(query)
```

```
        row = cursor.fetchone()
    if row:
        return HttpResponse(f'Welcome {row[1]}')
    return HttpResponse('Invalid login')
```

Step 2 - Live Attack (Do This Together in Class)

- 1. Open the login form: http://localhost:3000 (Node.js) or http://localhost:8000 (Django)
- 2. In the username field, type: ' OR '1'='1
- 3. Leave the password field empty, or type anything
- 4. Submit, and watch the app log you in as the first technician in the table, no password needed

VULNERABLE Think about it: what would happen if this were the OAS Bay admin login instead of a demo? Post your answer in the chat.

Step 3 - Fix It: Parameterised Queries

The fix is simple: never build SQL by joining strings together. Send the user's input separately, as a parameter, and let the database driver handle it safely. The database then always treats it as data, never as part of the instruction. Replace the Step 1 route with this version in the same file.

JAVASCRIPT / NODE.JS · server.js

```
// server.js - replace the /login route with this SAFE version
// placeholders (?) keep data separate from SQL
app.post('/login', (req, res) => {
  const { username, password } = req.body;
  const query = 'SELECT * FROM technicians WHERE username = ? AND
password = ?';
```

```
  db.get(query, [username, password], (err, row) => {
    if (row) return res.send('Welcome ' + row.username);
    return res.send('Invalid login');
  });
});
```

PYTHON / DJANGO · accounts/views.py

```
# accounts/views.py - replace login_view with this SAFE version
# the Django ORM parameterises the query automatically
@csrf_exempt
def login_view(request):
    username = request.POST.get('username')
    password = request.POST.get('password')
    technician = Technician.objects.filter(
        username=username, password=password
    ).first()
    if technician:
        return HttpResponse(f'Welcome {technician.username}')
    return HttpResponse('Invalid login')
```

FIX Node.js: the ? placeholders send username and password as data, never as SQL text. Django: the ORM builds a parameterised query for you automatically, this is one reason to always prefer the ORM over raw SQL.

Try the same ' OR '1'='1 attack again on the fixed version. It should now just fail to log in, exactly as it should.

4. Input Validation & Sanitisation

> **Definition:** Validation checks that input is the right shape before you use it, for example, is this actually a phone number? Sanitisation cleans or escapes input so it can't be misread as code or markup. Parameterised queries are your last line of defence. Use all three together, never just one.

The Three Layers of Defence

Layer	What It Catches	Can You Trust It Alone?
UI validation (HTML)	Obvious mistakes: empty fields, wrong format, while typing	No, it only runs in the browser, an attacker can bypass it completely
Backend validation	Rejects bad input before it ever touches the database or business logic	Mostly, this is your real defence, always validate on the server
Parameterised queries / ORM	Stops injection even if bad input somehow gets through validation	Yes, this is the safety net underneath everything else

Practical: A Simple, Validated Technician Login Form

We'll add both UI and backend validation to the same login form, so you can see authentication and validation working together.

UI Validation (HTML, Both Tracks)

```
<form method='POST' action='/login'>
```

```
  <input type='text' name='username' required minlength='3'
maxlength='20'
         pattern='[A-Za-z0-9_]+' title='Letters, numbers and
underscore only'>
  <input type='password' name='password' required minlength='6'>
  <button type='submit'>Log In</button>
</form>
<!-- required, minlength, maxlength and pattern are convenience
only. -->
<!-- An attacker can turn all of this off in DevTools. Never trust
it alone. -->
```

Backend Validation (The Part That Actually Matters)

JAVASCRIPT / NODE.JS · server.js

```
// server.js - replace the /login route again, validate before
touching the database
app.post('/login', (req, res) => {
  const { username, password } = req.body;

  if (!username || !password) {
    return res.status(400).send('Username and password are
required');
  }
  if (!/^[A-Za-z0-9_]{3,20}$/.test(username)) {
    return res.status(400).send('Invalid username format');
  }
  if (password.length < 6) {
    return res.status(400).send('Password must be at least 6
characters');
  }
```

```
  const query = 'SELECT * FROM technicians WHERE username = ? AND
password = ?';
  db.get(query, [username, password], (err, row) => {
    if (row) return res.send('Welcome ' + row.username);
    return res.send('Invalid login');
  });
});
```

PYTHON / DJANGO · accounts/forms.py

```
# accounts/forms.py  (new file) - Django validates and sanitises
for you when you use a Form
from django import forms

class LoginForm(forms.Form):
    username = forms.CharField(min_length=3, max_length=20)
    password = forms.CharField(min_length=6,
widget=forms.PasswordInput)
```

PYTHON / DJANGO · accounts/views.py

```
# accounts/views.py - replace login_view again, using the form to
validate
from .forms import LoginForm

@csrf_exempt
def login_view(request):
    form = LoginForm(request.POST)
    if not form.is_valid():
        return HttpResponse('Invalid input', status=400)

    username = form.cleaned_data['username']
    password = form.cleaned_data['password']
```

```
    technician = Technician.objects.filter(
        username=username, password=password
    ).first()
    if technician:
        return HttpResponse(f'Welcome {technician.username}')
    return HttpResponse('Invalid login')
```

FIX Notice the pattern in both languages: check the input is well-formed BEFORE you use it anywhere, and still use a parameterised query or the ORM underneath, even after validation. Validation and parameterisation are not substitutes for each other.

5. Recap

- A database stores data in tables made of rows and columns; SQL is how we query and change that data
- SQL Injection happens when user input is glued into a SQL string instead of kept separate as data
- ' OR '1'='1 works because it makes the WHERE clause always true
- Parameterised queries (Node.js placeholders, Django ORM) are the real fix, not string-building tricks
- UI validation is for user experience only, backend validation is your real defence, and parameterised queries are the safety net underneath both

Assignment: SQL Injection & Input Validation

> **Due before Day 3.** Work individually. Submit your before-and-after code, plus a short written explanation of what you found.

The Task

OAS Bay has a parts search feature: a customer or technician types a part name, and the system searches the parts table. Add this to the same project you built in class today (the same server.js, or the same Django project), so you're extending real working code, not starting from nothing.

> **Before you start:** add a parts table with columns id, name, price. Seed 4 to 5 rows yourself (e.g. 'Engine Oil 5W-30', 79000), the same way you created and seeded the technicians table earlier today.

Add this route to your project exactly as shown, it's deliberately vulnerable, that's your starting point.

JAVASCRIPT / NODE.JS · server.js

```
// GIVEN (vulnerable) - add to server.js, above app.listen()
app.get('/api/parts', (req, res) => {
  const term = req.query.name;
  const query = "SELECT * FROM parts WHERE name LIKE '%" + term +
"%'";
  db.all(query, (err, rows) => res.json(rows));
});
// Test it by visiting: http://localhost:3000/api/parts?name=oil
```

PYTHON / DJANGO · accounts/views.py

```
# GIVEN (vulnerable) - add to accounts/views.py, and wire it up in
accounts/urls.py
from django.http import JsonResponse

def search_parts(request):
    term = request.GET.get('name')
    query = "SELECT * FROM parts WHERE name LIKE '%" + term + "%'"
    with connection.cursor() as cursor:
        cursor.execute(query)
        rows = cursor.fetchall()
    return JsonResponse(list(rows), safe=False)

# accounts/urls.py: add  path('api/parts/', views.search_parts)
# Test it by visiting: http://localhost:8000/api/parts/?name=oil
```

You Must Submit

1. Proof of the attack: the exact payload you typed into the search field, and a screenshot or copy of the unexpected result it returned

2. A fixed version of the endpoint using a parameterised query (Node.js) or the Django ORM

3. UI validation added to the search input on the frontend (e.g. maxlength, an allowed-characters pattern)

4. Backend validation added before the query runs (reject empty input, enforce a max length, restrict characters to letters, numbers and spaces)

5. A 3 to 5 sentence explanation, in your own words, of why the original code was vulnerable and why your fix works

"Never trust a string you didn't write yourself."